Module 1 – Deep Learning Practicals (1–14)

All Sequential Keras models are built with `model = tf.keras.Sequential()` and `model.add(...)`. Run each practical separately. Install: `pip install tensorflow numpy matplotlib scikit-learn`.

Practical 1: Tensors and XOR

Code

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

a = tf.constant(10)
b = tf.constant([1, 2, 3])
c = tf.constant([[1, 2], [3, 4]])
print(a, b, c)
x = tf.constant([[10, 20], [30, 40]])
y = tf.constant([[2, 4], [5, 8]])
print("Add:", tf.add(x, y))
print("Subtract:", tf.subtract(x, y))
print("Multiply:", tf.multiply(x, y))
print("Divide:", tf.divide(x, y))
print("Reshape:", tf.reshape(x, [4, 1]))
print("Index:", x[0, 1].numpy())
print("Slice:", x[0:2, 0:1].numpy())
A = tf.constant([[1., 2.], [3., 4.]])
B = tf.constant([[5., 6.], [7., 8.]])
print("Matmul:", tf.matmul(A, B).numpy())
ev, evec = tf.linalg.eig(tf.constant([[4., 1.], [2., 3.]]))
print("Eigenvalues:", ev.numpy(), "Eigenvectors:", evec.numpy())

X = np.array([[0,0], [0,1], [1,0], [1,1]], dtype='float32')
Y = np.array([[0], [1], [1], [0]], dtype='float32')
model = tf.keras.Sequential()
model.add(tf.keras.layers.Input(shape=(2,)))
model.add(tf.keras.layers.Dense(8, activation='relu'))
model.add(tf.keras.layers.Dense(1, activation='sigmoid'))
model.compile(optimizer=tf.keras.optimizers.Adam(0.05),
loss='binary_crossentropy', metrics=['accuracy'])
h = model.fit(X, Y, epochs=1000, verbose=0)
p = model.predict(X, verbose=0)
print("XOR predictions:", (p >= 0.5).astype(int).ravel())
plt.plot(h.history['loss']); plt.title("XOR Loss"); plt.show()
```

Practical 2: Linear and Logistic Regression

Code

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Linear Regression
X = np.array([1, 2, 3, 4, 5], dtype='float32')
y = np.array([2, 4, 6, 8, 10], dtype='float32')
model = tf.keras.Sequential()
model.add(tf.keras.layers.Input(shape=(1,)))
model.add(tf.keras.layers.Dense(1))
model.compile(optimizer=tf.keras.optimizers.SGD(0.01), loss='mse')
h = model.fit(X, y, epochs=200, verbose=0)
print("Prediction for 6:", model.predict(np.array([[6.]], dtype='float32')))
plt.plot(h.history['loss']); plt.title("Linear Regression Loss"); plt.show()

# Logistic Regression
d = load_breast_cancer()
X, y = d.data.astype('float32'), d.target.astype('float32')
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
sc = StandardScaler()
Xtr = sc.fit_transform(Xtr); Xte = sc.transform(Xte)
model = tf.keras.Sequential()
model.add(tf.keras.layers.Input(shape=(Xtr.shape[1],)))
model.add(tf.keras.layers.Dense(1, activation='sigmoid'))
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model.fit(Xtr, ytr, epochs=30, batch_size=16)
print("Test:", model.evaluate(Xte, yte))
print("Predictions:", (model.predict(Xte[:5]) >= 0.5).astype(int).ravel())
```

Practical 3: Multilayer Perceptron (MLP)

Code

```
import tensorflow as tf
```

```
(Xtr, ytr), (Xte, yte) = tf.keras.datasets.mnist.load_data()
Xtr = Xtr.astype('float32') / 255.
Xte = Xte.astype('float32') / 255.
model = tf.keras.Sequential()
model.add(tf.keras.layers.Input(shape=(28, 28)))
model.add(tf.keras.layers.Flatten())
model.add(tf.keras.layers.Dense(128, activation='relu'))
model.add(tf.keras.layers.Dense(64, activation='relu'))
model.add(tf.keras.layers.Dense(10, activation='softmax'))
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
model.fit(Xtr, ytr, epochs=5, batch_size=32)
print(model.evaluate(Xte, yte))
```

Practical 4: CNN Classification and Binary Classification

Code

```
import tensorflow as tf
(Xtr, ytr), (Xte, yte) = tf.keras.datasets.mnist.load_data()
Xtr = Xtr.astype('float32').reshape(-1,28,28,1)/255.
Xte = Xte.astype('float32').reshape(-1,28,28,1)/255.
model = tf.keras.Sequential()
model.add(tf.keras.layers.Input(shape=(28,28,1)))
model.add(tf.keras.layers.Conv2D(32, 3, activation='relu'))
model.add(tf.keras.layers.MaxPooling2D())
model.add(tf.keras.layers.Conv2D(64, 3, activation='relu'))
model.add(tf.keras.layers.MaxPooling2D())
model.add(tf.keras.layers.Flatten())
model.add(tf.keras.layers.Dense(64, activation='relu'))
model.add(tf.keras.layers.Dense(10, activation='softmax'))
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
model.fit(Xtr, ytr, epochs=5, batch_size=64)
print("MNIST:", model.evaluate(Xte, yte))

# Binary classification: digit 0 versus digit 1
tr = (ytr == 0) | (ytr == 1); te = (yte == 0) | (yte == 1)
xb, yb = Xtr[tr], (ytr[tr] == 1).astype('float32')
xt, yt = Xte[te], (yte[te] == 1).astype('float32')
binary = tf.keras.Sequential()
binary.add(tf.keras.layers.Input(shape=(28,28,1)))
binary.add(tf.keras.layers.Conv2D(16, 3, activation='relu'))
binary.add(tf.keras.layers.MaxPooling2D())
binary.add(tf.keras.layers.Flatten())
binary.add(tf.keras.layers.Dense(32, activation='relu'))
binary.add(tf.keras.layers.Dense(1, activation='sigmoid'))
binary.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
binary.fit(xb, yb, epochs=3, batch_size=64)
print("Binary:", binary.evaluate(xt, yt))
```

Practical 5: Deep Feed Forward Network and Image Segmentation

Code

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

d = load_iris()
X, y = d.data.astype('float32'), d.target
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
sc = StandardScaler(); Xtr = sc.fit_transform(Xtr); Xte = sc.transform(Xte)
model = tf.keras.Sequential()
model.add(tf.keras.layers.Input(shape=(4,)))
model.add(tf.keras.layers.Dense(64, activation='relu'))
model.add(tf.keras.layers.Dense(32, activation='relu'))
model.add(tf.keras.layers.Dense(3, activation='softmax'))
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
model.fit(Xtr, ytr, epochs=40, verbose=0)
print("Iris test:", model.evaluate(Xte, yte))
print("Predicted classes:", model.predict(Xte[:5]).argmax(axis=1))

# Simple synthetic segmentation example (not real-world image masks)
images = np.random.rand(100,32,32,1).astype('float32')
masks = (images > 0.5).astype('float32')
seg = tf.keras.Sequential()
seg.add(tf.keras.layers.Input(shape=(32,32,1)))
seg.add(tf.keras.layers.Conv2D(16,3,padding='same',activation='relu'))
seg.add(tf.keras.layers.MaxPooling2D())
seg.add(tf.keras.layers.Conv2D(32,3,padding='same',activation='relu'))
seg.add(tf.keras.layers.UpSampling2D())
seg.add(tf.keras.layers.Conv2D(1,1,activation='sigmoid'))
seg.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
seg.fit(images, masks, epochs=3)
pred = seg.predict(images[:1])
plt.figure(figsize=(8,3))
plt.subplot(1,3,1); plt.imshow(images[0,:,:,0], cmap='gray'); plt.title('Input')
```

```
plt.subplot(1,3,2); plt.imshow(masks[0,:,:,0], cmap='gray'); plt.title('Mask')
plt.subplot(1,3,3); plt.imshow(pred[0,:,:,0], cmap='gray'); plt.title('Prediction')
plt.show()
```

Practical 6: Autoencoder and Denoising Autoencoder

Code

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
(Xtr, _), (Xte, _) = tf.keras.datasets.mnist.load_data()
Xtr = Xtr.astype('float32').reshape(-1,784)/255.
Xte = Xte.astype('float32').reshape(-1,784)/255.
Xtr_noise = np.clip(Xtr + 0.3*np.random.normal(size=Xtr.shape), 0, 1)
Xte_noise = np.clip(Xte + 0.3*np.random.normal(size=Xte.shape), 0, 1)
model = tf.keras.Sequential()
model.add(tf.keras.layers.Input(shape=(784,)))
model.add(tf.keras.layers.Dense(128, activation='relu'))
model.add(tf.keras.layers.Dense(32, activation='relu'))
model.add(tf.keras.layers.Dense(128, activation='relu'))
model.add(tf.keras.layers.Dense(784, activation='sigmoid'))
model.compile(optimizer='adam', loss='mse')
model.fit(Xtr_noise, Xtr, epochs=5, batch_size=256)
out = model.predict(Xte_noise[:5])
plt.figure(figsize=(10,4))
for i in range(5):
plt.subplot(2,5,i+1); plt.imshow(Xte_noise[i].reshape(28,28), cmap='gray'); plt.axis('off')
plt.subplot(2,5,i+6); plt.imshow(out[i].reshape(28,28), cmap='gray'); plt.axis('off')
plt.show()
```

Practical 7: Variational Autoencoder (VAE)

VAE requires a sampling layer and KL-divergence loss, so it uses a custom Keras training model. The decoder itself uses `model.add()`.

Code

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
(X, _), _ = tf.keras.datasets.mnist.load_data()
X = X.astype('float32').reshape(-1,784)/255.
latent = 2

inp = tf.keras.Input(shape=(784,))
h = tf.keras.layers.Dense(128, activation='relu')(inp)
mean = tf.keras.layers.Dense(latent)(h)
logvar = tf.keras.layers.Dense(latent)(h)
def sample(args):
m, lv = args
return m + tf.exp(0.5*lv)*tf.random.normal(tf.shape(m))
z = tf.keras.layers.Lambda(sample)([mean, logvar])
encoder = tf.keras.Model(inp, [mean, logvar, z])

decoder = tf.keras.Sequential()
decoder.add(tf.keras.layers.Input(shape=(latent,)))
decoder.add(tf.keras.layers.Dense(128, activation='relu'))
decoder.add(tf.keras.layers.Dense(784, activation='sigmoid'))

class VAE(tf.keras.Model):
def __init__(self, enc, dec):
super().__init__(); self.enc=enc; self.dec=dec
def train_step(self, data):
with tf.GradientTape() as tape:
m, lv, z = self.enc(data)
rec = self.dec(z)
rec_loss = tf.reduce_mean(tf.reduce_sum(
tf.keras.backend.binary_crossentropy(data, rec), axis=1))
kl = -0.5*tf.reduce_mean(tf.reduce_sum(
1+lv-tf.square(m)-tf.exp(lv), axis=1))
loss = rec_loss + kl
grads = tape.gradient(loss, self.trainable_variables)
self.optimizer.apply_gradients(zip(grads, self.trainable_variables))
return {'loss': loss}

vae = VAE(encoder, decoder)
vae.compile(optimizer='adam')
vae.fit(X, epochs=5, batch_size=128)
generated = decoder.predict(tf.random.normal((5,latent)))
plt.figure(figsize=(10,2))
for i in range(5):
plt.subplot(1,5,i+1); plt.imshow(generated[i].reshape(28,28), cmap='gray'); plt.axis('off')
plt.show()
```

Practical 8: Generative Adversarial Network (GAN)

GAN needs two networks and alternating training. Generator and discriminator are each built with `model.add()`.

Code

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
(X, _), _ = tf.keras.datasets.mnist.load_data()
X = X.astype('float32').reshape(-1,784)/127.5 - 1.

generator = tf.keras.Sequential()
generator.add(tf.keras.layers.Input(shape=(100,)))
generator.add(tf.keras.layers.Dense(128, activation='relu'))
generator.add(tf.keras.layers.Dense(256, activation='relu'))
generator.add(tf.keras.layers.Dense(784, activation='tanh'))

discriminator = tf.keras.Sequential()
discriminator.add(tf.keras.layers.Input(shape=(784,)))
discriminator.add(tf.keras.layers.Dense(256, activation='relu'))
discriminator.add(tf.keras.layers.Dense(128, activation='relu'))
discriminator.add(tf.keras.layers.Dense(1, activation='sigmoid'))
discriminator.compile(optimizer='adam', loss='binary_crossentropy')

discriminator.trainable = False
gan = tf.keras.Sequential()
gan.add(generator)
gan.add(discriminator)
gan.compile(optimizer='adam', loss='binary_crossentropy')

for epoch in range(1000):
idx = np.random.randint(0, len(X), 32)
real = X[idx]
noise = tf.random.normal((32,100))
fake = generator.predict(noise, verbose=0)
discriminator.trainable = True
discriminator.train_on_batch(real, np.ones((32,1)))
discriminator.train_on_batch(fake, np.zeros((32,1)))
discriminator.trainable = False
gan.train_on_batch(tf.random.normal((32,100)), np.ones((32,1)))

samples = generator.predict(tf.random.normal((5,100)))
plt.figure(figsize=(10,2))
for i in range(5):
plt.subplot(1,5,i+1); plt.imshow(samples[i].reshape(28,28), cmap='gray'); plt.axis('off')
plt.show()
```

Practical 9: LSTM Caption Word Prediction (simplified)

This demonstrates next-word prediction from sample captions. Full image captioning requires CNN image features paired with captions.

Code

```
import tensorflow as tf
import numpy as np
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
captions = ['a dog is running', 'a cat is sleeping',
'a man is playing', 'a dog is playing']
tok = Tokenizer(); tok.fit_on_texts(captions)
seqs = tok.texts_to_sequences(captions)
X, y = [], []
for seq in seqs:
for i in range(1,len(seq)):
X.append(seq[:i]); y.append(seq[i])
maxlen = max(map(len,X))
X = pad_sequences(X, maxlen=maxlen); y = np.array(y)
model = tf.keras.Sequential()
model.add(tf.keras.layers.Embedding(len(tok.word_index)+1, 32))
model.add(tf.keras.layers.LSTM(64))
model.add(tf.keras.layers.Dense(len(tok.word_index)+1, activation='softmax'))
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
model.fit(X, y, epochs=200, verbose=0)
def next_word(text):
seq = pad_sequences(tok.texts_to_sequences([text]), maxlen=maxlen)
idx = int(np.argmax(model.predict(seq, verbose=0)[0]))
return tok.index_word.get(idx, '<unknown>')
print('Next word:', next_word('a dog'))
```

Practical 10: RNN Sequence Prediction

Code

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
```

```
data = np.sin(np.arange(1000)*0.02).astype('float32')
X, y = [], []
for i in range(len(data)-10):
X.append(data[i:i+10]); y.append(data[i+10])
X = np.array(X).reshape(-1,10,1); y = np.array(y)
model = tf.keras.Sequential()
model.add(tf.keras.layers.Input(shape=(10,1)))
model.add(tf.keras.layers.SimpleRNN(50, activation='tanh'))
model.add(tf.keras.layers.Dense(1))
model.compile(optimizer='adam', loss='mse')
model.fit(X, y, epochs=10, batch_size=32)
pred = model.predict(X[:100])
plt.plot(y[:100], label='Actual'); plt.plot(pred, label='Predicted')
plt.legend(); plt.show()
```

Practical 11: RNN vs CNN Digit Recognition

MNIST contains digits (0–9), not alphabet characters.

Code

```
import tensorflow as tf
import numpy as np
(Xtr,ytr),(Xte,yte)=tf.keras.datasets.mnist.load_data()
Xtr=Xtr.astype('float32')/255.; Xte=Xte.astype('float32')/255.
rnn=tf.keras.Sequential()
rnn.add(tf.keras.layers.Input(shape=(28,28)))
rnn.add(tf.keras.layers.SimpleRNN(64))
rnn.add(tf.keras.layers.Dense(10,activation='softmax'))
rnn.compile(optimizer='adam',loss='sparse_categorical_crossentropy',metrics=['accuracy'])
rnn.fit(Xtr,ytr,epochs=3,batch_size=64)
rnn_acc=rnn.evaluate(Xte,yte,verbose=0)[1]

cnn=tf.keras.Sequential()
cnn.add(tf.keras.layers.Input(shape=(28,28,1)))
cnn.add(tf.keras.layers.Conv2D(32,3,activation='relu'))
cnn.add(tf.keras.layers.MaxPooling2D())
cnn.add(tf.keras.layers.Flatten())
cnn.add(tf.keras.layers.Dense(10,activation='softmax'))
cnn.compile(optimizer='adam',loss='sparse_categorical_crossentropy',metrics=['accuracy'])
cnn.fit(Xtr[...,None],ytr,epochs=3,batch_size=64)
cnn_acc=cnn.evaluate(Xte[...,None],yte,verbose=0)[1]
print('RNN Accuracy:',rnn_acc)
print('CNN Accuracy:',cnn_acc)
```

Practical 12: Regularization – L2, Dropout and Early Stopping

Code

```
import tensorflow as tf
import matplotlib.pyplot as plt
(Xtr,ytr),(Xte,yte)=tf.keras.datasets.fashion_mnist.load_data()
Xtr=Xtr.astype('float32')/255.; Xte=Xte.astype('float32')/255.
model=tf.keras.Sequential()
model.add(tf.keras.layers.Input(shape=(28,28)))
model.add(tf.keras.layers.Flatten())
model.add(tf.keras.layers.Dense(128,activation='relu',
kernel_regularizer=tf.keras.regularizers.l2(0.001)))
model.add(tf.keras.layers.Dropout(0.3))
model.add(tf.keras.layers.Dense(64,activation='relu'))
model.add(tf.keras.layers.Dropout(0.2))
model.add(tf.keras.layers.Dense(10,activation='softmax'))
model.compile(optimizer='adam',loss='sparse_categorical_crossentropy',metrics=['accuracy'])
stop=tf.keras.callbacks.EarlyStopping(monitor='val_loss',patience=3,restore_best_weights=True)
h=model.fit(Xtr,ytr,epochs=20,validation_split=0.2,callbacks=[stop],batch_size=64)
print(model.evaluate(Xte,yte))
plt.plot(h.history['loss'],label='Train'); plt.plot(h.history['val_loss'],label='Validation')
plt.legend(); plt.show()
```

Practical 13: Boltzmann Machine (simplified educational example)

A classical Boltzmann Machine is not a simple Sequential neural network; this demonstrates stochastic binary sampling and weight updates.

Code

```
import numpy as np
data=np.array([[1,1,0],[1,0,1],[0,1,1],[0,0,1]],dtype=float)
n=data.shape[1]
W=np.random.normal(0,0.1,(n,n))
W=(W+W.T)/2
np.fill_diagonal(W,0)
bias=np.zeros(n)
def sigmoid(x): return 1/(1+np.exp(-x))
for epoch in range(500):
for v in data:
prob=sigmoid(np.dot(v,W)+bias)
```

```
W += 0.01*(np.outer(v,v)-np.outer(prob,prob))
bias += 0.01*(v-prob)
W=(W+W.T)/2
np.fill_diagonal(W,0)
state=np.array([1.,0.,1.])
for _ in range(20):
prob=sigmoid(np.dot(state,W)+bias)
state=(np.random.rand(n)<prob).astype(float)
print('Weights:\n',W)
print('Generated binary sample:',state)
```

Practical 14: Restricted Boltzmann Machine (RBM) using CD-1

Code

```
import numpy as np
data=np.array([[1,1,0,0],[1,0,1,0],[0,1,0,1],[0,0,1,1]],dtype=float)
nv,nh=4,2
W=np.random.normal(0,0.1,(nv,nh))
vb=np.zeros(nv); hb=np.zeros(nh)
lr=0.1
def sigmoid(x): return 1/(1+np.exp(-x))
for epoch in range(1000):
for v0 in data:
# Positive phase
h0p=sigmoid(np.dot(v0,W)+hb)
h0=(np.random.rand(nh)<h0p).astype(float)
# Negative phase (one Gibbs step)
v1p=sigmoid(np.dot(h0,W.T)+vb)
v1=(np.random.rand(nv)<v1p).astype(float)
h1p=sigmoid(np.dot(v1,W)+hb)
# CD-1 parameter updates
W += lr*(np.outer(v0,h0p)-np.outer(v1,h1p))
vb += lr*(v0-v1)
hb += lr*(h0p-h1p)
features=sigmoid(np.dot(data,W)+hb)
print('Learned weights:\n',W)
print('Encoded hidden features:\n',features)
```

Result / Important Notes

All 14 practicals are included. For Sequential models, layers are added using `model.add()`.

- Practical 5 uses generated toy masks to demonstrate segmentation architecture; real segmentation needs real images and matching masks.

- Practical 9 is an LSTM next-word caption demonstration, not full CNN+LSTM image captioning.

- Practical 11 compares RNN and CNN on MNIST handwritten digits.

- Practical 13 is a simplified Boltzmann-style educational example.

- VAE and GAN require custom/multiple-network training logic; BM and RBM use sampling/update equations, so forcing them into one `model.add()` stack would be incorrect.